

Теория Параллелизма

Отчёт

Лабораторная работа № 6.
Уравнение теплопроводности

Выполнила: Ильиных Татьяна 24940

03.06.2026

Цель работы

Реализовать численное решение уравнения теплопроводности в двумерной области методом Якоби. Перенести вычисления на графический процессор с помощью директив OpenACC, выполнить оптимизацию кода и провести сравнительный анализ производительности на центральном и графическом процессорах для различных размеров сетки (128×128 , 256×256 , 512×512 , 1024×1024).

Используемый компилятор

NVIDIA HPC SDK - pgc++ версии 23.11-0

Ключи компиляции:

- `std=c++11` - использование стандарта C++11
- `O3` - максимальная оптимизация
- `acc` - включение поддержки OpenACC
- `Minfo=all` - вывод подробной информации о распараллеливании
- Линковка: `-lboost_program_options` - библиотека для парсинга аргументов командной строки

Используемый профилировщик

NVIDIA Nsight Systems

Как производили замер времени работы

Замер времени выполнения программы производился с использованием библиотеки `<chrono>` стандарта C++11. Алгоритм замера:

- Фиксация начального момента времени: `auto start_time = std::chrono::steady_clock::now();`
- Выполнение основного итерационного цикла (метод Якоби)
- Фиксация конечного момента времени: `auto end_time = std::chrono::steady_clock::now();`

Использованный тип часов: `std::chrono::steady_clock` - монотонные часы, которые не подвержены корректировке системного времени, что обеспечивает максимальную точность измерений.

Что именно измерялось: время, затраченное на выполнение основного итерационного процесса (метод Якоби), включая вычисление новой сетки и проверку ошибки сходимости. Инициализация граничных условий и вывод результатов в измеряемое время не включались.

Выполнение на CPU

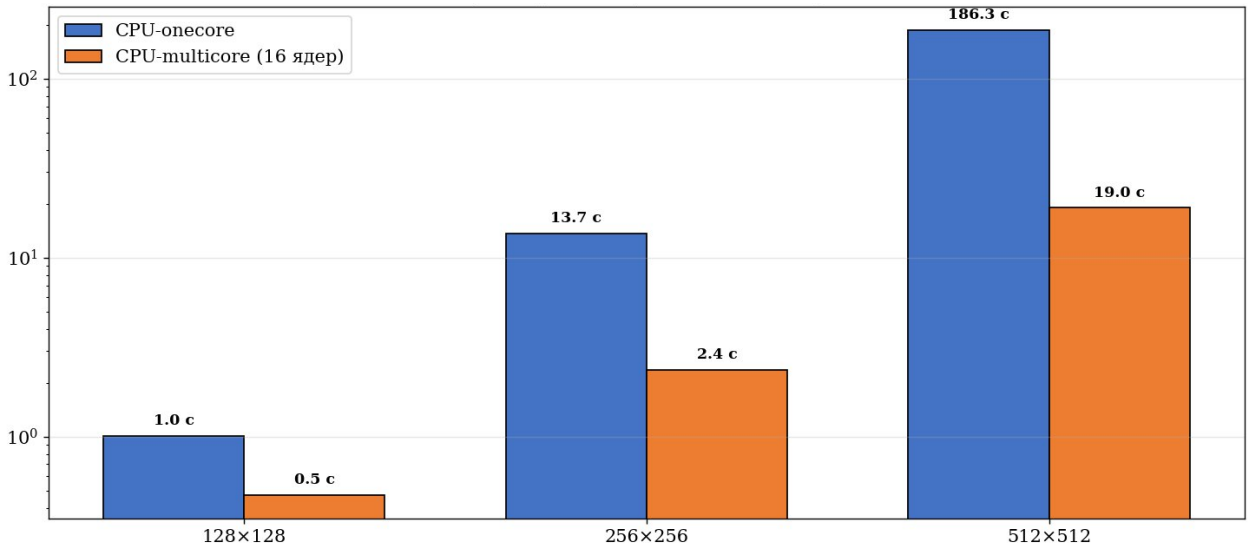
CPU-onecore

Размер сетки	Время выполнения, сек	Точность	Количество итераций
128×128	1.006	9.9998e-07	30 074
256×256	13.651	9.9993e-07	102 885
512×512	186.314	9.99984e-07	339 599

CPU-multicore

Размер сетки	Время выполнения, сек	Точность	Количество итераций
128×128	0.471	9.9998e-07	30 074
256×256	2.361	9.9993e-07	102 885
512×512	19.004	9.99984e-07	339 599
1024×1024	182.942	1.36929e-06	1 000 000

Диаграмма сравнения время работы CPU-one и CPU-multi



Выполнение на GPU

Этапы оптимизации на сетке 512*512

Этап №	Время выполнения, сек	Точность	Максимальное количество итераций	Комментарии
--------	-----------------------	----------	----------------------------------	-------------

1	661.6	9.99984e-07	1 000 000	Базовая версия.
2	13.095	9.99984e-07	1 000 000	Добавлен data region: #pragma acc data copy(...). Данные копируются один раз в начале и остаются на GPU на протяжении всех итераций. Значительное ускорение за счет уменьшения накладных расходов на передачу данных.
3	7.831	9.99965e-07	1 000 000	Периодическая проверка ошибки: Добавлен параметр check_iter. Вычисление ошибки сходимости (reduction(max:error)) выполняется не на каждой итерации, а каждые 100 итераций. Это уменьшает количество дорогостоящих операций редукции и синхронизаций.
4	7.815	9.99965e-07	1 000 000	Добавлен vector_length(256): В директиву OpenACC добавлен параметр vector_length(256), который оптимизирует загрузку GPU, устанавливая оптимальное количество потоков в векторе (256). Это позволяет эффективнее использовать регистры и кэш-память GPU.

Профилирование Nsight Systems

Этап № 1

```
[6/8] Executing 'cuda_gpu_kern_sum' stats report
```

Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
66.7	732885	100	7328.9	7296.5	7231	9280	202.4	_660_cpp_main_148_gpu
33.3	366200	100	3662.0	3648.0	3647	4384	74.1	_660_cpp_main_148_gpu__red

```
[7/8] Executing 'cuda_gpu_mem_time_sum' stats report
```

Time (%)	Total Time (ns)	Count	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Operation
66.4	64745477	300	215818.3	320381.0	1344	351644	151936.1	[CUDA memcpy Host-to-Device]
33.5	32676072	300	108920.2	162270.0	1824	168350	75795.6	[CUDA memcpy Device-to-Host]
0.1	137083	100	1370.8	1376.0	1343	1408	13.4	[CUDA memset]

```
[8/8] Executing 'cuda_gpu_mem_size_sum' stats report
```

Total (MB)	Count	Avg (MB)	Med (MB)	Min (MB)	Max (MB)	StdDev (MB)	Operation
419.436	300	1.398	2.097	0.000	2.097	0.990	[CUDA memcpy Host-to-Device]
419.431	300	1.398	2.097	0.000	2.097	0.990	[CUDA memcpy Device-to-Host]
0.001	100	0.000	0.000	0.000	0.000	0.000	[CUDA memset]

Этап № 4

[6/8] Executing 'cuda_gpu_kern_sum' stats report

Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
97.8	523360	99	5286.5	5280.0	5184	6240	111.9	_663_cpp_main_174_gpu
1.6	8576	1	8576.0	8576.0	8576	8576	0.0	_663_cpp_main_157_gpu
0.6	3232	1	3232.0	3232.0	3232	3232	0.0	_663_cpp_main_157_gpu__red

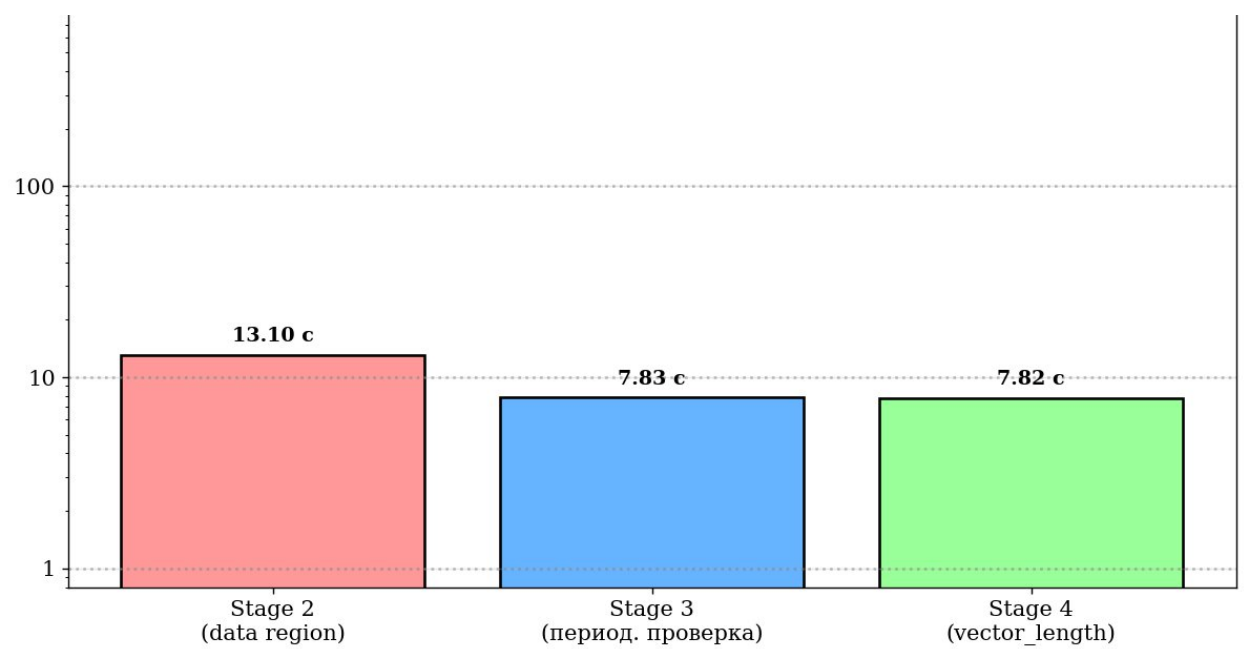
[7/8] Executing 'cuda_gpu_mem_time_sum' stats report

Time (%)	Total Time (ns)	Count	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Operation
70.6	788218	102	7727.6	1376.0	1344	326044	45111.9	[CUDA memcpy Host-to-Device]
29.3	327261	3	109087.0	162366.0	2144	162751	92615.6	[CUDA memcpy Device-to-Host]
0.1	1376	1	1376.0	1376.0	1376	1376	0.0	[CUDA memset]

[8/8] Executing 'cuda_gpu_mem_size_sum' stats report

Total (MB)	Count	Avg (MB)	Med (MB)	Min (MB)	Max (MB)	StdDev (MB)	Operation
4.200	102	0.041	0.000	0.000	2.097	0.292	[CUDA memcpy Host-to-Device]
4.194	3	1.398	2.097	0.000	2.097	1.211	[CUDA memcpy Device-to-Host]
0.000	1	0.000	0.000	0.000	0.000	0.000	[CUDA memset]

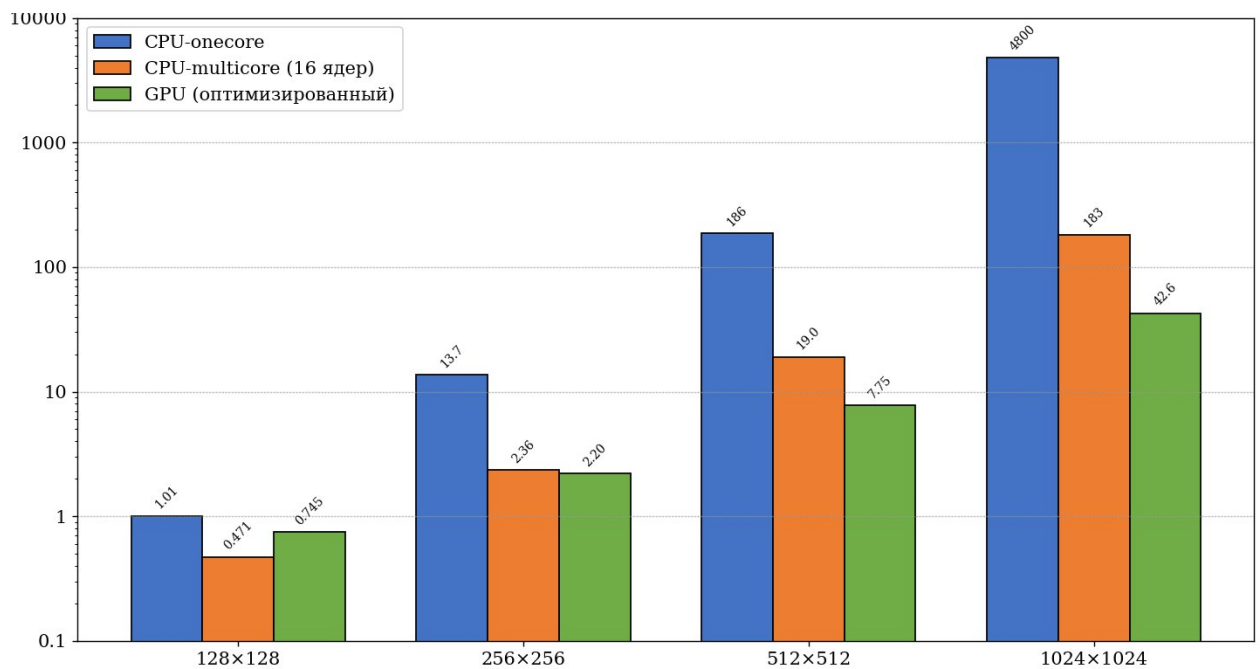
Диаграмма оптимизации



GPU - оптимизированный вариант

Размер сетки	Время выполнения, сек	Точность	Количество итераций
128×128	0.745	9.92056e-07	30 100
256×256	2.202	9.98793e-07	102 900
512×512	7.754	9.99965e-07	339 600
1024×1024	42.617	1.36929e-06	1 000 000

Сравнительная диаграмма CPU-one, CPU-multi, GPU



Вывод сетки 10×10

```

Финальная сетка (10x10):
10.0000 11.1111 12.2222 13.3333 14.4444 15.5556 16.6667 17.7778 18.8889 20.0000
11.1111 12.2222 13.3333 14.4444 15.5556 16.6667 17.7778 18.8889 20.0000 21.1111
12.2222 13.3333 14.4444 15.5556 16.6667 17.7778 18.8889 20.0000 21.1111 22.2222
13.3333 14.4444 15.5556 16.6667 17.7778 18.8889 20.0000 21.1111 22.2222 23.3333
14.4444 15.5556 16.6667 17.7778 18.8889 20.0000 21.1111 22.2222 23.3333 24.4444
15.5556 16.6667 17.7778 18.8889 20.0000 21.1111 22.2222 23.3333 24.4444 25.5556
16.6667 17.7778 18.8889 20.0000 21.1111 22.2222 23.3333 24.4444 25.5556 26.6667
17.7778 18.8889 20.0000 21.1111 22.2222 23.3333 24.4444 25.5556 26.6667 27.7778
18.8889 20.0000 21.1111 22.2222 23.3333 24.4444 25.5556 26.6667 27.7778 28.8889
20.0000 21.1111 22.2222 23.3333 24.4444 25.5556 26.6667 27.7778 28.8889 30.0000

```

Вывод сетки 13×13

```

Финальная сетка (13x13):
10.0000 10.8333 11.6667 12.5000 13.3333 14.1667 15.0000 15.8333 16.6667 17.5000 18.3333 19.1667 20.0000
10.8333 11.6667 12.5000 13.3333 14.1667 15.0000 15.8333 16.6667 17.5000 18.3333 19.1667 20.0000 20.8333
11.6667 12.5000 13.3333 14.1667 15.0000 15.8333 16.6667 17.5000 18.3333 19.1667 20.0000 20.8333 21.6667
12.5000 13.3333 14.1667 15.0000 15.8333 16.6667 17.5000 18.3333 19.1667 20.0000 20.8333 21.6667 22.5000
13.3333 14.1667 15.0000 15.8333 16.6667 17.5000 18.3333 19.1667 20.0000 20.8333 21.6667 22.5000 23.3333
14.1667 15.0000 15.8333 16.6667 17.5000 18.3333 19.1667 20.0000 20.8333 21.6667 22.5000 23.3333 24.1667
15.0000 15.8333 16.6667 17.5000 18.3333 19.1667 20.0000 20.8333 21.6667 22.5000 23.3333 24.1667 25.0000
15.8333 16.6667 17.5000 18.3333 19.1667 20.0000 20.8333 21.6667 22.5000 23.3333 24.1667 25.0000 25.8333
16.6667 17.5000 18.3333 19.1667 20.0000 20.8333 21.6667 22.5000 23.3333 24.1667 25.0000 25.8333 26.6667
17.5000 18.3333 19.1667 20.0000 20.8333 21.6667 22.5000 23.3333 24.1667 25.0000 25.8333 26.6667 27.5000
18.3333 19.1667 20.0000 20.8333 21.6667 22.5000 23.3333 24.1667 25.0000 25.8333 26.6667 27.5000 28.3333
19.1667 20.0000 20.8333 21.6667 22.5000 23.3333 24.1667 25.0000 25.8333 26.6667 27.5000 28.3333 29.1667
20.0000 20.8333 21.6667 22.5000 23.3333 24.1667 25.0000 25.8333 26.6667 27.5000 28.3333 29.1667 30.0000

```

Вывод

На малых сетках (128×128) GPU не дает преимущества из-за накладных расходов на запуск ядер.

На средних и больших сетках GPU значительно выигрывает. Ускорение растет с увеличением размера задачи.

Основное узкое место в неоптимизированной версии - избыточное копирование данных на каждой итерации.

1. Data region (#pragma acc data copy) позволило сократить количество копирований данных между CPU и GPU.
2. Периодическая проверка ошибки - уменьшила число дорогостоящих операций редукции. Дополнительное ускорение - 1.7 раза.
3. Vector_length(256) - также дало незначительное улучшение (0.2%).